Université Mohammed Premier
École Nationale de l'Intelligence Artificielle
et du Digital — Berkane

COMPTE RENDU DE TRAVAUX PRATIQUES

BIG DATA

Cluster HDFS avec Docker Compose

TP 2

RÉALISÉ PAR **Zakariae El Haddouchi**

FILIÈRE Intelligence Artificielle (cycle ingénieur)

MODULE Big Data

PROFESSEUR **Phd. AL ALAMI NABIL**

CODE SOURCE

github.com/Eng-ZAKARIAE-AI/bigdata-hadoop

Année universitaire 2026 – 2027

Exercice 1 | Présentation du cluster HDFS

Ce TP vise à déployer un cluster Hadoop HDFS minimal (1 NameNode + 2 DataNodes) en utilisant Docker Compose. Le cluster repose sur l'image officielle `apache/hadoop:3.5.0`, avec une configuration personnalisée montée en volume.

1.1 Structure du projet

- `Dockerfile`
- `docker-compose.yml`
- `config/` : `core-site.xml`, `hdfs-site.xml`
- `data/` : `namenode_name/`, `datanode1/`, `datanode2/`
- `run-hdfs.sh`
- `starter.sh`
- `hadoop-wrapper.sh`
- `README.md`

Description des composants.

- `Dockerfile` : construit l'image `tp2-hadoop:3.5.0` à partir de `apache/hadoop:3.5.0`, copie les fichiers de configuration Hadoop et définit `hdfs` comme point d'entrée.
- `docker-compose.yml` : orchestre les trois services `namenode`, `datanode1` et `datanode2` sur le réseau `hdfs-net`.
- `config/core-site.xml` : définit le système de fichiers par défaut (`fs.defaultFS` = `hdfs://namenode:8020`).
- `config/hdfs-site.xml` : configure la réplication, la taille des blocs et les répertoires de stockage.
- `data/` : répertoires locaux montés dans les conteneurs pour persister les métadonnées et les blocs HDFS.

Exercice 2 | Configuration HDFS

`config/core-site.xml`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:8020</value>
  </property>
</configuration>
```

`config/hdfs-site.xml`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>dfs.replication</name>
```

```
    <value>2</value>
  </property>
  <property>
    <name>dfs.blocksize</name>
    <value>8388608</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/data/hdfs/name</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/data/hdfs/data</value>
  </property>
</configuration>
```

Interprétation des paramètres.

- `dfs.replication = 2` : chaque bloc est répliqué sur 2 DataNodes, ce qui correspond au nombre de DataNodes du cluster.
- `dfs.blocksize = 8 388 608` : la taille d'un bloc HDFS est fixée à 8 Mio (au lieu des 128 Mio par défaut), ce qui facilite les tests pédagogiques avec de petits fichiers.
- `dfs.namenode.name.dir = /data/hdfs/name` : les métadonnées du NameNode (namespace, images fsimage, edit logs) sont persistées dans ce répertoire monté depuis l'hôte.
- `dfs.datanode.data.dir = /data/hdfs/data` : les blocs de données sont stockés dans ce répertoire sur chaque DataNode.
- `fs.defaultFS = hdfs://namenode:8020` : l'URI HDFS par défaut pointe vers le NameNode sur le port RPC 8020.

Exercice 3 | Docker Compose : orchestration des services

`docker-compose.yml`

```yaml
services:
  namenode:
    image: tp2-hadoop:3.5.0
    container_name: tp2-namenode
    hostname: namenode
    user: hadoop
    command: ["namenode"]
    ports:
      - "9870:9870" # UI web NameNode
      - "8020:8020" # Port RPC
    volumes:
      - ./config:/opt/hadoop/etc/hadoop:ro
      - ./data/namenode_name:/data/hdfs/name
    networks: [hdfs-net]

  datanode1:
    image: tp2-hadoop:3.5.0
    container_name: tp2-datanode1
    hostname: datanode1
```

```
    user: hadoop
    command: ["datanode"]
    depends_on: [namenode]
    volumes:
      - ./config:/opt/hadoop/etc/hadoop:ro
      - ./data/datanode1:/data/hdfs/data
    networks: [hdfs-net]

  datanode2:
    image: tp2-hadoop:3.5.0
    container_name: tp2-datanode2
    hostname: datanode2
    user: hadoop
    command: ["datanode"]
    depends_on: [namenode]
    volumes:
      - ./config:/opt/hadoop/etc/hadoop:ro
      - ./data/datanode2:/data/hdfs/data
    networks: [hdfs-net]

networks:
  hdfs-net:
```

Analyse de l'orchestration. Le service `namenode` expose deux ports : `9870` pour l'interface web d'administration HDFS et `8020` pour les appels RPC des clients et DataNodes. Les deux DataNodes dépendent du NameNode (`depends_on`) et partagent le même réseau Docker `hdfs-net`, ce qui permet la résolution DNS mutuelle entre les conteneurs (`namenode`, `datanode1`, `datanode2`).

Le montage `./config:/opt/hadoop/etc/hadoop:ro` écrase les fichiers de configuration inclus dans l'image Docker par ceux du projet, garantissant que le cluster utilise bien notre configuration personnalisée. Les répertoires `./data/*` assurent la persistance des métadonnées et des blocs entre les redémarrages.

Exercice 4 | Dockerfile : construction de l'image

`Dockerfile`

```
FROM apache/hadoop:3.5.0

COPY config/core-site.xml /opt/hadoop/etc/hadoop/core-site.xml
COPY config/hdfs-site.xml /opt/hadoop/etc/hadoop/hdfs-site.xml

ENTRYPOINT ["/opt/hadoop/bin/hdfs"]
```

Explication. L'image de base `apache/hadoop:3.5.0` fournit une installation complète de Hadoop. Les deux fichiers de configuration sont copiés dans le répertoire `/opt/hadoop/etc/hadoop/` de l'image, remplaçant les fichiers par défaut. L'écrasement du point d'entrée par `/opt/hadoop/bin/hdfs` permet de contourner le script `envtoconf.py` défaillant de l'image officielle : chaque conteneur

reçoit directement l'argument `namenode` ou `datanode` via la commande `hdfs`.

Exercice 5 | Démarrage et vérification du cluster

5.1 Construction de l'image

```
docker build -t tp2-hadoop:3.5.0 .
```

5.2 Formatage du NameNode (initialisation uniquement)

```
docker compose run --rm namenode namenode -format
```

Cette étape crée les métadonnées initiales du NameNode dans `data/namenode_name/`. Elle n'est à exécuter qu'une seule fois, lors de la première initialisation du cluster.

5.3 Démarrage du cluster

```
docker compose up -d
```

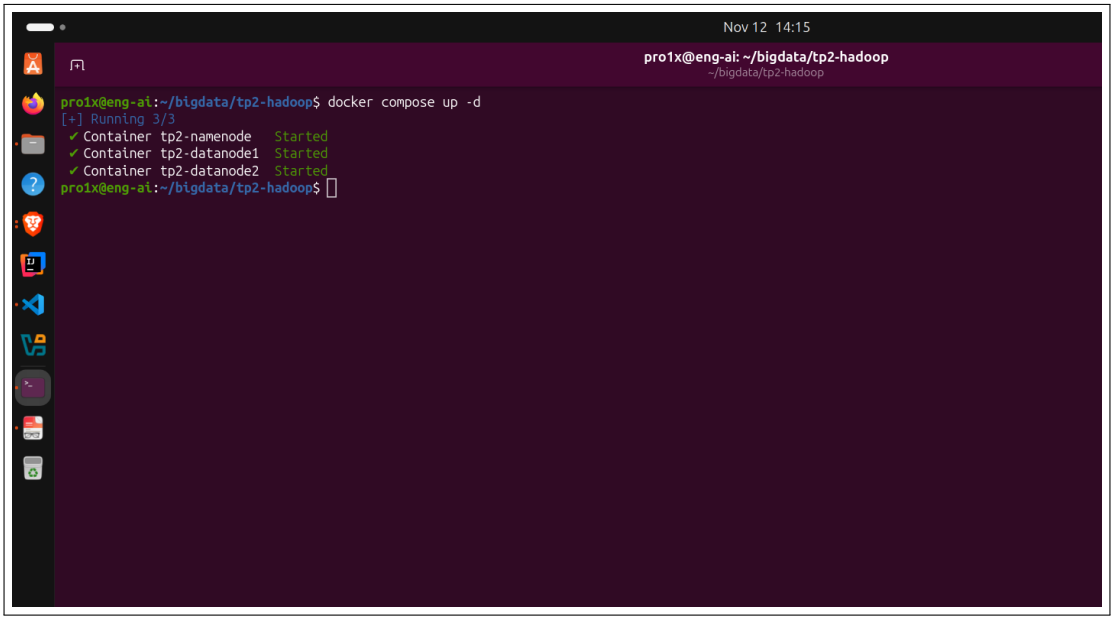


FIG. 1 : Démarrage du cluster HDFS avec Docker Compose

5.4 Vérification de l'état

```
docker compose ps
docker compose logs -f namenode
docker compose logs -f datanode1 datanode2
```

L'interface web du NameNode est accessible à l'adresse `http://localhost:9870`.

Exercice 6 | Opérations HDFS courantes

6.1 Lister le répertoire racine HDFS

```
docker compose exec namenode hdfs dfs -ls /
```

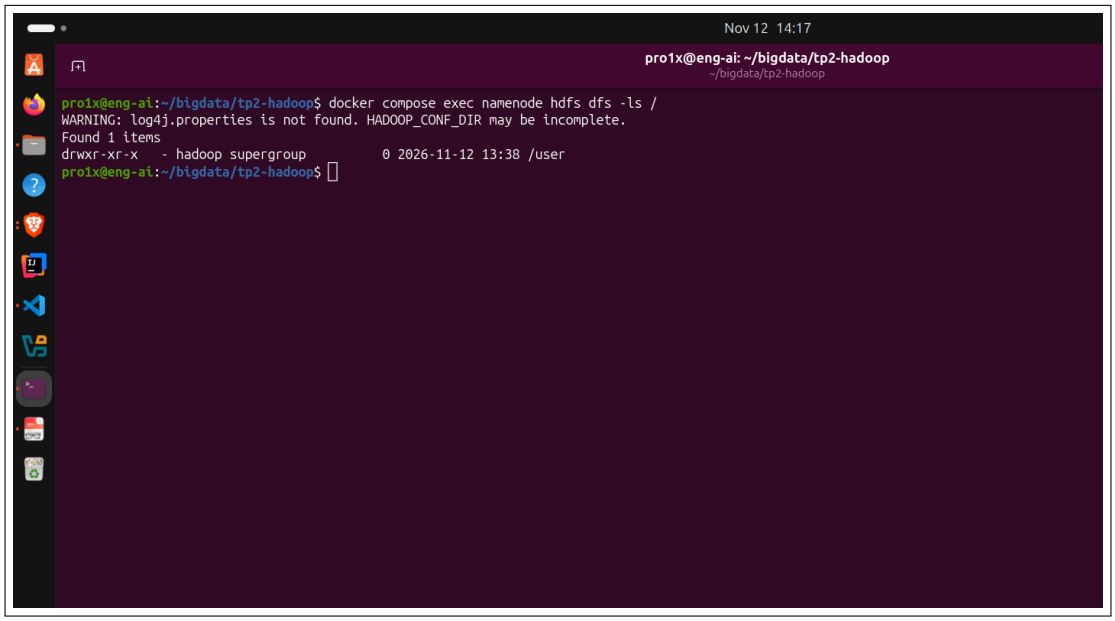


FIG. 2 : Liste du répertoire racine HDFS

6.2 Créer un répertoire et y déposer un fichier

```
docker compose exec namenode hdfs dfs -mkdir -p /tp2/data
echo "Bonjour HDFS depuis Docker Compose" > local.txt
docker compose exec namenode hdfs dfs -put local.txt /tp2/data/
```

6.3 Vérifier la réplication des blocs

```
docker compose exec namenode hdfs fsck /tp2/data/local.txt -files -blocks
```

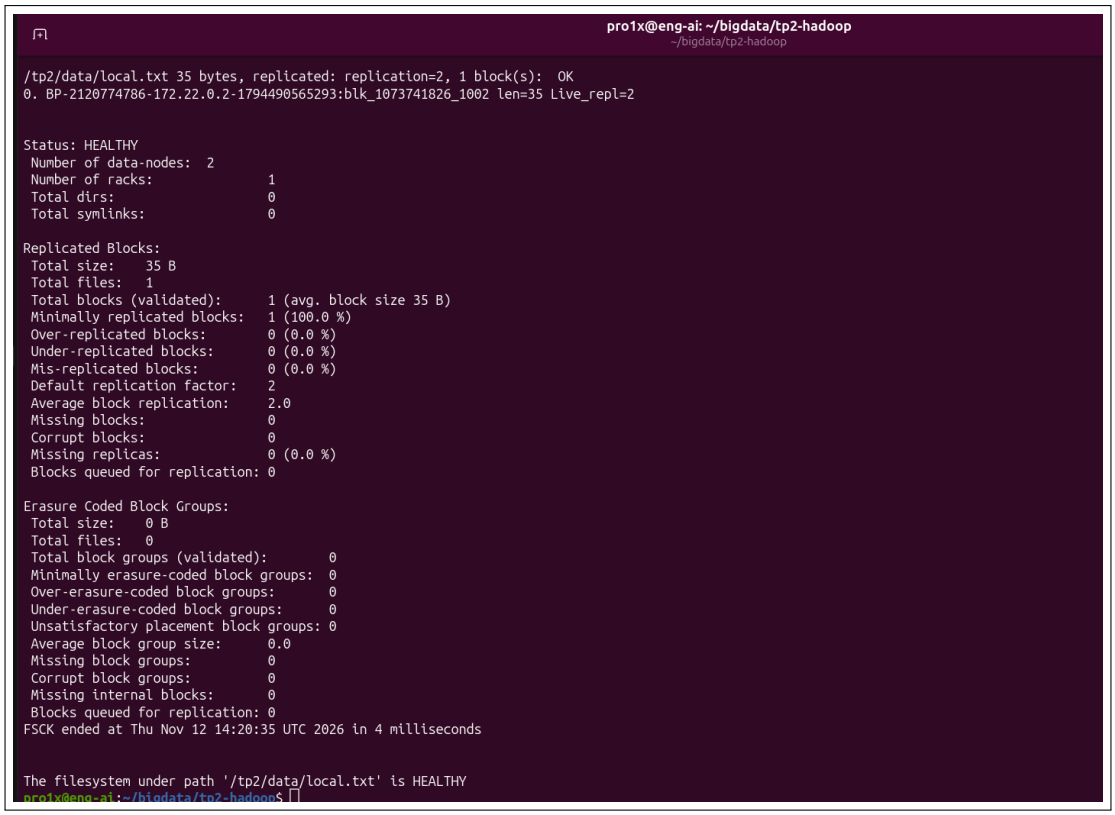


FIG. 3 : Vérification de la réplication des blocs avec fsck

Interprétation. La commande `fsck` confirme que le fichier `local.txt` a bien été découpé en blocs et répliqué sur les deux DataNodes conformément à `dfs.replication = 2`. Chaque bloc apparaît avec deux répliques, ce qui valide le bon fonctionnement de la configuration HDFS du cluster.

Exercice 7 | Arrêt du cluster

```
1 docker compose down
```

Cette commande arrête et supprime les conteneurs, mais les données persistent dans les répertoires locaux `data/` montés en volume. Un redémarrage avec `docker compose up -d` restaurera le cluster sans nécessiter de nouveau formatage.

Exercice 8 | Conclusion

Ce TP a permis de mettre en pratique les concepts fondamentaux de Hadoop HDFS à travers un déploiement Docker Compose :

- Comprendre l'architecture NameNode / DataNode et le rôle de chaque composant.
- Configurer HDFS via `core-site.xml` et `hdfs-site.xml` (réplication, taille de bloc, répertoires de stockage).
- Orchestrer un cluster multi-nœuds avec Docker Compose (réseau partagé, volumes de

persistance, dépendances entre services).
— Exécuter des opérations HDFS courantes (`mkdir`, `put`, `ls`, `fsck`) depuis le conteneur NameNode.